

COURSE STRUCTURE

Course Code	BCA20120				
Course Category	Program Major				
Course Title	Software Engineering				
Teaching Scheme	Lectures	Tutorials	Laboratory / Practical	Project	Total
Weekly load hours	3	-	-	-	3
Credits	3	-	-	-	3
Assessment Scheme Code	TT1				

Pre-requisites:

Understanding of Computer Science Fundamentals, Software development

Course Objectives:

1. To understand the knowledge and skills necessary for effective software development and management.
2. To Understanding Software Engineering
3. To develop the ability to gather, analyze, and document software requirements

Course Outcomes:

1. To articulate the specific knowledge, skills, and abilities that students are expected to gain by the end of the course.
2. Requirements Engineering:
3. Elicit, analyze, and document software requirements effectively, considering both functional and non-functional aspects.
4. System Design Proficiency:
5. Design software systems using appropriate architectural styles, patterns, and object-oriented design principles.

Course Contents:

Unit 1: Introduction to Software Engineering[2]

Definition and goals of software engineering, Software development life cycle models, Roles and responsibilities in software development teams, Ethical and professional issues in software engineering

Unit 2: Software Processes: [3]

Software process models , Process activities, Coping with change, Process improvement

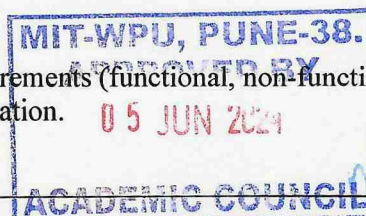
Unit 3: Agile Methodologies[5]

Agile methods, Agile development techniques, Agile project management, Scaling agile methods

Unit 4: Software Requirements[5]

Requirement elicitation and analysis, Types of requirements (functional, non-functional) Use cases and scenarios, Requirement specification, and documentation.

Unit 5: System Design[5]



Architectural design, Component-level design, Object-oriented design principles, Design patterns
Design documentation

Unit 6: Implementation and Coding[3]

Programming languages and paradigms, Code organization and structure, Coding standards and best practices, Code reviews and inspections.

Unit 5: Software Testing[3]

Testing levels (unit, integration, system, acceptance), Testing types (functional, non-functional), Test planning and strategy, Test automation.

Unit 6: Software Maintenance and Evolution[4]

Software maintenance and its importance, Software reengineering and reverse engineering, Version control systems, and Code refactoring.

Unit 7: Software Project Management:[5]

Project planning and scheduling, Risk management, Team collaboration and communication
Agile and Scrum methodologies.

Unit 8: Software Quality Assurance[5]

Quality metrics and measurement, Continuous integration and continuous delivery (CI/CD), DevOps practices.

Unit 9: Emerging Trends in Software Engineering[5]

Cloud computing and its impact on software development, Artificial intelligence in software engineering, Internet of Things (IoT) and software engineering

Learning Resources:

Text Books/Reference Books:

1. SOFTWARE ENGINEERING: A PRACTITIONER'S APPROACH, 7TH EDITION"
2. Software Engineering: A PRACTITIONER'S APPROACH, By Rajiv Chopra
3. GitHub Learning Lab-Provides hands-on, project-based learning using GitHub.
4. LeetCode-Platform for practicing coding problems and preparing for technical interviews.

Web Resources :

Weblinks:

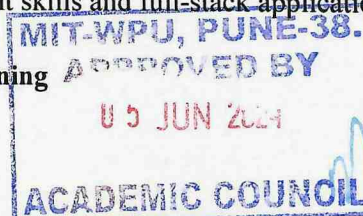
1. Coursera - Software Engineering Specialization (offered by University of British Columbia)
 - A comprehensive specialization covering various aspects of software engineering.
2. edX - Software Development and Testing MicroMasters Program (offered by the University of British Columbia)
 - A series of courses covering software development, testing, and best practices.
3. Udacity - Full Stack Web Developer Nanodegree
 - Focuses on web development skills and full-stack applications.

MOOCs: Online courses for self-learning

Courses by NPTEL and MIT
Open Courseware etc

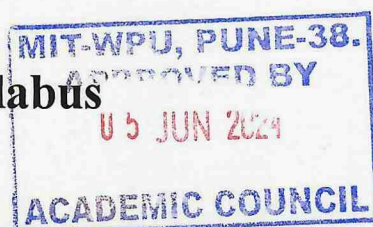
Pedagogy:

- Participative Learning,



- Discussions,
- Problem solving,
- Assignments.

Syllabus



M. Bechu
EP. Kholi